

1. Algorithms for Each Activity

Activity 1: Solid Square Pattern

1. Start the function `solidSquare()`.
2. Declare integer variables `n`, `i`, and `j`.
3. Prompt the user to enter the number of rows/columns `n`.
4. Read `n`.
5. Start outer loop `i` from 1 to `n` (controls rows):
 - Start inner loop `j` from 1 to `n` (controls columns):
 - Print asterisk `"*"`.
 - Print a newline character `" "` to move to the next row.
6. End function.

Activity 2: Solid Rectangular Pattern

1. Start the function `solidRectangle()`.
2. Declare integer variables `m` (rows), `n` (columns), `i`, and `j`.
3. Prompt the user to enter the number of rows `m` and columns `n`.
4. Read `m` and `n`.
5. Start outer loop `i` from 1 to `m`:
 - Start inner loop `j` from 1 to `n`:
 - Print asterisk `"*"`.
 - Print a newline character `" "`.
6. End function.

Activity 3: Row-Number Pattern

1. Start the function `rowNumberPattern()`.
2. Declare integer variables `m`, `n`, `i`, and `j`.
3. Prompt the user to enter the number of rows `m` and columns `n`.
4. Read `m` and `n`.
5. Start outer loop `i` from 1 to `m`:
 - Start inner loop `j` from 1 to `n`:
 - Print the current row number `i` followed by a space.
 - Print a newline character `" "`.
6. End function.

Activity 4: Column-Number Pattern

1. Start the function `columnNumberPattern()`.
2. Declare integer variables `m`, `n`, `i`, and `j`.
3. Prompt the user to enter the number of rows `m` and columns `n`.
4. Read `m` and `n`.

5. Start outer loop *i* from 1 to *m*:
 - Start inner loop *j* from 1 to *n*:
 - Print the current column number *j* followed by a space.
 - Print a newline character " \n".
6. End function.

2. Test-Case Tables

Activity 1: Solid Square Pattern

Test Case ID	Input (n)	Expected Output	Status
TC_SQ_01	n = 4	4 rows, each containing 4 asterisks (****)	Pass
TC_SQ_02	n = 2	2 rows, each containing 2 asterisks (**)	Pass

Activity 2: Solid Rectangular Pattern

Test Case ID	Input (m, n)	Expected Output	Status
TC_REC_01	m = 3, n = 5	3 rows, each containing 5 asterisks (*****)	Pass
TC_REC_02	m = 2, n = 4	2 rows, each containing 4 asterisks (****)	Pass

Activity 3: Row-Number Pattern

Test Case ID	Input (m, n)	Expected Output	Status
TC_ROW_01	m = 4, n = 4	4 rows, row 1 with '1 1 1 1', row 2 with '2 2 2 2', etc.	Pass
TC_ROW_02	m = 3, n = 3	3 rows, row 1 with '1 1 1', row 2 with '2 2 2', row 3 with '3 3 3'	Pass

Activity 4: Column-Number Pattern

Test Case ID	Input (m, n)	Expected Output	Status
TC_COL_01	m = 4, n = 4	4 identical rows, each containing '1 2 3 4'	Pass
TC_COL_02	m = 3, n = 5	3 identical rows, each containing '1 2 3 4 5'	Pass

3. Complete C Program

The program uses parameterless functions with no return value (`void`) for each pattern as required.

```

#include <stdio.h>

/* Function prototypes */
void solidSquare(void);
void solidRectangle(void);
void rowNumberPattern(void);
void columnNumberPattern(void);

int main() {
    int choice;

    do {
        printf("
=====
");
        printf("          PATTERN MENU
");
        printf("=====
");
        printf("1. Solid square pattern
");
        printf("2. Solid rectangular pattern
");
        printf("3. Row-number pattern
");
        printf("4. Column-number pattern
");
        printf("5. Exit
");
        printf("-----
");
        printf("Enter your choice (1-5): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                solidSquare();
                break;
            case 2:
                solidRectangle();
                break;
            case 3:
                rowNumberPattern();
                break;
            case 4:
                columnNumberPattern();
                break;
            case 5:
                printf("
Exiting program. Thank you!
");
                break;
            default:
                printf("
Invalid choice! Please select between 1 and 5.
");
        }
    } while (choice != 5);
}

```

```

    return 0;
}

/* 1. Function to print Solid Square Pattern */
void solidSquare(void) {
    int n, i, j;

    printf("
--- Solid Square Pattern ---
");
    printf("Enter the number of rows/columns (n): ");
    scanf("%d", &n);

    printf("
Output:
");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("* ");
        }
        printf("
");
    }
}

/* 2. Function to print Solid Rectangular Pattern */
void solidRectangle(void) {
    int m, n, i, j;

    printf("
--- Solid Rectangular Pattern ---
");
    printf("Enter number of rows (m): ");
    scanf("%d", &m);
    printf("Enter number of columns (n): ");
    scanf("%d", &n);

    printf("
Output:
");
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++) {
            printf("* ");
        }
        printf("
");
    }
}

/* 3. Function to print Row-Number Pattern */
void rowNumberPattern(void) {
    int m, n, i, j;

    printf("
--- Row-Number Pattern ---
");
    printf("Enter number of rows (m): ");
    scanf("%d", &m);
    printf("Enter number of columns (n): ");
    scanf("%d", &n);

```

```

        printf("
Output:
");
        for (i = 1; i <= m; i++) {
            for (j = 1; j <= n; j++) {
                printf("%d ", i);
            }
            printf("
");
        }
    }

/* 4. Function to print Column-Number Pattern */
void columnNumberPattern(void) {
    int m, n, i, j;

    printf("
--- Column-Number Pattern ---
");
    printf("Enter number of rows (m): ");
    scanf("%d", &m);
    printf("Enter number of columns (n): ");
    scanf("%d", &n);

    printf("
Output:
");
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++) {
            printf("%d ", j);
        }
        printf("
");
    }
}

```

4. Sample Execution Outputs

1. Solid Square Pattern (n = 4)

```

* * * *
* * * *
* * * *
* * * *

```

2. Solid Rectangular Pattern (m = 3, n = 5)

```

* * * * *
* * * * *
* * * * *

```

3. Row-Number Pattern ($m = 4$, $n = 4$)

1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4

4. Column-Number Pattern ($m = 4$, $n = 4$)

1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4